Conteúdo de apoio

O uso do Git pode ser feito de várias formas. As IDEs e editores de código mais modernos, por exemplo, já trazem plugins e soluções nativas para utilização desse sistema de controle de versão. Nesse vídeo, no entanto, para apresentar os principais comandos independentemente do ambiente de desenvolvimento que você faz uso, optamos pelo terminal do sistema operacional.

Assim, após instalar o Git em sua máquina, os dois primeiros comandos a executar no prompt são:

```
git config --global user.name "Nome do usuario"
git config --global user.email e-mail@exemplo.com
```

O Git pede uma identificação, para que possa sinalizar o autor das mudanças realizadas no repositório. Logo após, no diretório em que você pretende criar ou já criou o projeto, execute:

```
git init
```

Isso é o que precisamos para inicializar o repositório e criar a pasta oculta .git. Agora, nosso diretório também é o que chamamos na linguagem Git de working area, o diretório de trabalho.

Qualquer arquivo criado/modificado neste diretório será monitorado pelo Git. Para que possamos adicionar esse arquivo em nosso repositório, primeiro precisamos adicioná-lo ao stage, o que é feito com o comando:

```
git add NomeDoArquivo.extensão
```

Dessa forma o arquivo estará referenciado nesse espaço intermediário. A partir disso, para confirmar as mudanças e adicionar o novo arquivo ao repositório, precisamos executar:

```
git commit -m “Mensagem descritiva das mudanças realizadas.”
```

É válido destacar a importância da mensagem especificado junto com o commit. É recomendado que a partir dela o programador saiba tudo o que foi feito naquela versão.

Outro conceito importante do Git é o conceito de branch. É aconselhado criar um branch sempre que você desejar testar alguma nova API/Biblioteca/Framework ou começar a criar uma nova funcionalidade no projeto. Quando inicializamos o repositório, automaticamente ele cria o branch master. Esse é o branch principal do projeto. Quando estamos trabalhando em equipe, principalmente, o ideal é que esse branch receba apenas versões do código com as funcionalidades já testadas e avaliadas.

Para começar a criar uma nova funcionalidade, como já mencionado, o ideal é criar um novo branch. Ao fazer isso, lembre-se de dar um nome autoexplicativo. Para criar o branch, execute:

```
git branch nomeDoBranch
```

Ao fazer isso, é como se criássemos uma cópia de todos os arquivos do nosso projeto. A diferença é que essa cópia é gerenciada pelo Git. Portanto, você não verá arquivos duplicados. A partir disso, para acessar o novo branch, o nosso novo ramo, execute no terminal:

```
git checkout nomeDoBranch
```

Logo após, todas as mudanças realizadas a partir de agora estarão acessíveis apenas nesse branch. Portanto, se você criar um ou mais arquivos, ao voltar para o branch master (com o comando git checkout master) esses arquivos não serão encontrados, pois nesse ramo esses arquivos não existem.

Agora que entendemos como Git funciona, podemos resumir o fluxo básico de trabalho com ele à sequência de imagens abaixo: